CSCI 544 - Homework Assignment 2

Sentiment Analysis with Word Embeddings and Neural Networks

Name: Sohail Haresh Gidwani
USC ID: 7321203258
Library: PyTorch

Report

1. Dataset Generation (5 points)

A balanced dataset of **250,000** Amazon product reviews was constructed by randomly sampling 50,000 reviews per star rating (1 through 5). Ternary sentiment labels were assigned as follows: ratings > 3 map to **class 1 (positive)**, ratings < 3 map to **class 2 (negative)**, and rating = 3 maps to **class 3 (neutral)**. This yields 100,000 positive, 100,000 negative, and 50,000 neutral reviews. The dataset was split 80/20 into 200,000 training and 50,000 testing samples. Data cleaning followed the same pipeline as HW1: contraction expansion, HTML tag removal, lowercasing, non-alphabetic character removal, stopword removal, and lemmatization.

2. Word Embedding (30 points)

(a) Pretrained Word2Vec (10 points):
The `word2vec-google-news-300` model (3M vocabulary, 300-d vectors) was loaded. Two semantic similarity tests were performed:

- *king - man + woman* correctly returns **queen** (similarity 0.7118).
- *similarity(excellent, outstanding)* = **0.5567**.

(b) Custom Word2Vec (20 points):
A Word2Vec model was trained on the review corpus (vector_size=300, window=11, min_count=10, seed=42). Vocabulary size: **14,281** words.

- *king - man + woman* returns domain-specific words (latin, gender) since royalty terms are rare in product reviews.
- *similarity(excellent, outstanding)* = **0.7500** — higher than pretrained, showing the custom model captures sentiment-related similarities better within the review domain.

Conclusion: The pretrained model excels at general-purpose analogies due to its massive training corpus. The custom model, despite its small vocabulary, captures domain-specific sentiment relationships more effectively — sentiment words like "excellent" and "outstanding" appear in similar review contexts, producing tighter similarity scores.

3. Simple Models (20 points)

Binary classification (positive vs negative) using average Word2Vec features:

Model	Pretrained W2V	Custom W2V
Perceptron	0.7571	**0.7931**
SVM	0.8160	**0.8410**

The custom Word2Vec features outperform pretrained features for both models. SVM outperforms Perceptron in both settings, consistent with SVM's stronger generalization through margin maximization. The custom model's advantage stems from its domain-specific training — word embeddings are tuned to the sentiment distinctions present in product reviews.

4. Feedforward Neural Networks (25 points)

(a) MLP with Average W2V Features (10 points):

Configuration	Pretrained W2V	Custom W2V
Binary	0.8460	**0.8632**
Ternary	0.6865	**0.6995**

(b) MLP with Concatenated W2V Features (15 points):

Configuration	Pretrained W2V	Custom W2V
Binary	0.7467	**0.7530**
Ternary	0.5645	**0.5754**

Key findings:

- Average W2V features significantly outperform concatenated features across all settings. Averaging produces a smooth, fixed-size representation of overall review sentiment, while concatenation is sensitive to word order and zero-padding for short reviews.
- The MLP with average features (0.8632 binary) substantially improves over the simple SVM (0.8410), demonstrating the benefit of non-linear decision boundaries.

- Ternary classification is considerably harder than binary, as the neutral class overlaps with both positive and negative sentiments.

5. Convolutional Neural Networks (20 points)

Configuration	Pretrained W2V	Custom W2V
Binary	**0.8561**	0.8513
Ternary	0.6921	**0.6964**

The CNN achieves the **highest binary accuracy (0.8561)** among all models by capturing local n-gram patterns through its convolutional filters. For ternary classification, CNN (0.6964) and MLP Avg (0.6995) perform comparably. Notably, for CNN the pretrained model slightly outperforms custom on binary — the pretrained model's broad vocabulary helps capture diverse word patterns in sequential features.

Summary of All 16 Accuracy Values

#	Model	Embedding	Task	Accuracy
1	Perceptron	Pretrained	Binary	0.7571
2	Perceptron	Custom	Binary	0.7931
3	SVM	Pretrained	Binary	0.8160
4	SVM	Custom	Binary	0.8410
5	MLP (Avg)	Pretrained	Binary	0.8460
6	MLP (Avg)	Pretrained	Ternary	0.6865
7	MLP (Avg)	Custom	Binary	0.8632
8	MLP (Avg)	Custom	Ternary	0.6995
9	MLP (Concat)	Pretrained	Binary	0.7467
10	MLP (Concat)	Pretrained	Ternary	0.5645
11	MLP (Concat)	Custom	Binary	0.7530
12	MLP (Concat)	Custom	Ternary	0.5754
13	CNN	Pretrained	Binary	0.8561
14	CNN	Pretrained	Ternary	0.6921
15	CNN	Custom	Binary	0.8513
16	CNN	Custom	Ternary	0.6964

The complete Jupyter Notebook with all code, outputs, and visualizations follows below.

In [1]:
```
import pandas as pd
import numpy as np
import re
import gc
import warnings
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, TensorDataset, DataLoader
import nltk
from bs4 import BeautifulSoup
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatri
from sklearn.manifold import TSNE
from gensim.models import Word2Vec
import gensim.downloader as api
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.dpi'] = 120

warnings.filterwarnings('ignore')
nltk.download('wordnet', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)

# Set seeds for reproducibility
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

```
Using device: cpu
```

Question 1: Dataset Generation (5 points)

Load the Amazon reviews dataset, build a balanced dataset of 250K reviews (50K per rating), and create ternary sentiment labels.

In [2]:
```
# Load the Amazon reviews dataset
df = pd.read_csv('amazon_reviews_us_Office_Products_v1_00.tsv', sep='\t', or
df = df[['review_body', 'star_rating']].copy()
df = df.dropna()
df['star_rating'] = pd.to_numeric(df['star_rating'], errors='coerce')
```

```
df = df.dropna()
df['star_rating'] = df['star_rating'].astype(int)

print(f"Total reviews loaded: {len(df)}")
print("\nRating distribution:")
print(df['star_rating'].value_counts().sort_index())
```

```
Total reviews loaded: 2640080

Rating distribution:
star_rating
1     306967
2     138381
3     193680
4     418348
5    1582704
Name: count, dtype: int64
```

In [3]:

```
# Sample 50K reviews per rating to build balanced 250K dataset
balanced_parts = []
for rating in range(1, 6):
    subset = df[df['star_rating'] == rating]
    sampled = subset.sample(n=50000, random_state=SEED)
    balanced_parts.append(sampled)

df_balanced = pd.concat(balanced_parts, ignore_index=True)

# Free the original large dataframe to save memory
del df, balanced_parts, subset, sampled
gc.collect()

# Create ternary labels: >3 -> 1 (positive), <3 -> 2 (negative), =3 -> 3 (ne
def assign_sentiment(rating):
    if rating > 3:
        return 1  # positive
    elif rating < 3:
        return 2  # negative
    else:
        return 3  # neutral

df_balanced['sentiment'] = df_balanced['star_rating'].apply(assign_sentiment

print(f"Balanced dataset size: {len(df_balanced)}")
print("\nSentiment distribution:")
print(f"  Class 1 (positive, rating > 3): {(df_balanced['sentiment'] == 1).s
print(f"  Class 2 (negative, rating < 3): {(df_balanced['sentiment'] == 2).s
print(f"  Class 3 (neutral, rating = 3):  {(df_balanced['sentiment'] == 3).s
```

```
Balanced dataset size: 250000

Sentiment distribution:
  Class 1 (positive, rating > 3): 100000
  Class 2 (negative, rating < 3): 100000
  Class 3 (neutral, rating = 3):  50000
```

In [4]:

```
# --- Data Cleaning (same approach as HW1) ---
```

```python
contractions_dict = {
    "won't": "will not", "can't": "cannot", "n't": " not", "'re": " are",
    "'s": " is", "'d": " would", "'ll": " will", "'ve": " have", "'m": " am"
    "let's": "let us", "it's": "it is", "i'm": "i am", "you're": "you are",
    "he's": "he is", "she's": "she is", "we're": "we are", "they're": "they
    "i've": "i have", "you've": "you have", "we've": "we have", "they've": "
    "i'd": "i would", "you'd": "you would", "he'd": "he would", "she'd": "sh
    "we'd": "we would", "they'd": "they would", "i'll": "i will", "you'll":
    "he'll": "he will", "she'll": "she will", "we'll": "we will", "they'll":
    "isn't": "is not", "aren't": "are not", "wasn't": "was not", "weren't":
    "haven't": "have not", "hasn't": "has not", "hadn't": "had not",
    "doesn't": "does not", "don't": "do not", "didn't": "did not",
    "wouldn't": "would not", "shouldn't": "should not", "couldn't": "could r
    "mightn't": "might not", "mustn't": "must not", "needn't": "need not",
    "what's": "what is", "who's": "who is", "where's": "where is",
    "when's": "when is", "why's": "why is", "how's": "how is",
    "that's": "that is", "there's": "there is", "here's": "here is",
    "gonna": "going to", "gotta": "got to", "wanna": "want to",
    "kinda": "kind of", "sorta": "sort of", "lemme": "let me", "gimme": "giv
}
contractions_pattern = re.compile(
    '({})'.format('|'.join(contractions_dict.keys())), flags=re.IGNORECASE
)

def expand_contractions(text):
    def replace(match):
        return contractions_dict.get(match.group(0).lower(), match.group(0))
    return contractions_pattern.sub(replace, text)

def clean_text(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()
    text = BeautifulSoup(text, "html.parser").get_text()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = expand_contractions(text)
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    words = text.split()
    words = [w for w in words if w.lower() not in stop_words]
    words = [lemmatizer.lemmatize(w) for w in words]
    return ' '.join(words)

print("Cleaning and preprocessing reviews...")
df_balanced['cleaned'] = df_balanced['review_body'].apply(clean_text)
df_balanced['processed'] = df_balanced['cleaned'].apply(preprocess_text)
print("Done.")

# Show a few examples
for i in range(3):
```

```
    print(f"\nSample {i+1}:")
    print(f"  Original : {df_balanced['review_body'].iloc[i]}")
    print(f"  Processed: {df_balanced['processed'].iloc[i]}")
```

```
Cleaning and preprocessing reviews...
Done.

Sample 1:
  Original : I purchased these tabs on a whim to put up some movie posters.
I used four of the 1 by 1 inch tabs for each poster and they did not do the
job.  The posters stayed up for a few days and then one right after the othe
r just started falling down.  I guess I'll just have to go back to my clear
thumbtacks.  This product is not even worth a penny.
  Processed: purchased tab whim put movie poster used four inch tab poster j
ob poster stayed day one right started falling guess go back clear thumbtack
product even worth penny

Sample 2:
  Original : Returned it.  Too much garbage involved in setup.
  Processed: returned much garbage involved setup

Sample 3:
  Original : My upholstered living room chairs aren't particularly big, but
the chair cover wasn't big enough to cover all the fabric on the back of the
chair.  Also, I thought these would be chair shaped, but turns out they're j
ust shaped like a trash bag.
  Processed: upholstered living room chair particularly big chair cover big
enough cover fabric back chair also thought would chair shaped turn shaped l
ike trash bag
```

In [5]:

```
# Tokenize the processed reviews for Word2Vec training
df_balanced['tokens'] = df_balanced['processed'].apply(lambda x: x.split())

# Drop columns no longer needed to save memory
df_balanced.drop(columns=['review_body', 'star_rating', 'cleaned'], inplace=
gc.collect()

# 80/20 train-test split
train_df, test_df = train_test_split(df_balanced, test_size=0.2, random_stat
train_df = train_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print(f"Training set size: {len(train_df)}")
print(f"Testing set size:  {len(test_df)}")
```

```
Training set size: 200000
Testing set size:  50000
```

Question 2: Word Embedding (30 points)

Question 2(a): Pretrained Word2Vec (10 points)

Load the pretrained `word2vec-google-news-300` model and demonstrate semantic similarity with two examples.

In [6]:
```
# Load the pretrained Google News Word2Vec model
print("Loading pretrained word2vec-google-news-300 model (this may take a fe
pretrained_w2v = api.load('word2vec-google-news-300')
print(f"Loaded. Vocabulary size: {len(pretrained_w2v)}")
```

```
Loading pretrained word2vec-google-news-300 model (this may take a few minut
es)...
Loaded. Vocabulary size: 3000000
```

In [7]:
```
# Semantic similarity Example 1: king - man + woman ≈ queen
result_1 = pretrained_w2v.most_similar(positive=['king', 'woman'], negative=
print("Example 1 (Pretrained): king - man + woman =")
for word, sim in result_1:
    print(f"  {word}: {sim:.4f}")

# Semantic similarity Example 2: excellent ~ outstanding
sim_score = pretrained_w2v.similarity('excellent', 'outstanding')
print(f"\nExample 2 (Pretrained): similarity('excellent', 'outstanding') = {

# Additional check: most similar to 'excellent'
result_2 = pretrained_w2v.most_similar('excellent', topn=5)
print("\nWords most similar to 'excellent' (Pretrained):")
for word, sim in result_2:
    print(f"  {word}: {sim:.4f}")
```

```
Example 1 (Pretrained): king - man + woman =
  queen: 0.7118
  monarch: 0.6190
  princess: 0.5902
  crown_prince: 0.5499
  prince: 0.5377

Example 2 (Pretrained): similarity('excellent', 'outstanding') = 0.5567

Words most similar to 'excellent' (Pretrained):
  terrific: 0.7410
  superb: 0.7063
  exceptional: 0.6815
  fantastic: 0.6803
  good: 0.6443
```

Question 2(b): Custom-Trained Word2Vec (20 points)

Train a Word2Vec model on the review dataset with embedding size 300, window size 11, and minimum word count 10.

In [8]:
```
# Train a custom Word2Vec model on the full balanced dataset
all_tokens = df_balanced['tokens'].tolist()

print("Training custom Word2Vec model...")
custom_w2v_model = Word2Vec(
```

```
    sentences=all_tokens,
    vector_size=300,
    window=11,
    min_count=10,
    seed=SEED,
    workers=1  # single worker for reproducibility
)
custom_w2v = custom_w2v_model.wv

# Free training model and token list to save memory
del custom_w2v_model, all_tokens
gc.collect()

print(f"Custom model trained. Vocabulary size: {len(custom_w2v)}")
```

```
Training custom Word2Vec model...
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Custom model trained. Vocabulary size: 14281
```

In [9]:

```
# Semantic similarity Example 1: king - man + woman (Custom model)
try:
    result_1_custom = custom_w2v.most_similar(positive=['king', 'woman'], ne
    print("Example 1 (Custom): king - man + woman =")
    for word, sim in result_1_custom:
        print(f"  {word}: {sim:.4f}")
except KeyError as e:
    print(f"Example 1 (Custom): Word not in vocabulary - {e}")

# Semantic similarity Example 2: excellent ~ outstanding (Custom model)
try:
    sim_score_custom = custom_w2v.similarity('excellent', 'outstanding')
    print(f"\nExample 2 (Custom): similarity('excellent', 'outstanding') = {
except KeyError as e:
    print(f"\nExample 2 (Custom): Word not in vocabulary - {e}")

# Most similar to 'excellent' in custom model
try:
    result_2_custom = custom_w2v.most_similar('excellent', topn=5)
    print("\nWords most similar to 'excellent' (Custom):")
    for word, sim in result_2_custom:
        print(f"  {word}: {sim:.4f}")
except KeyError as e:
    print(f"\nWord not in vocabulary - {e}")
```

```
Example 1 (Custom): king - man + woman =
  latin: 0.5130
  gender: 0.5018
  american: 0.5007
  webster: 0.4905
  modo: 0.4851

Example 2 (Custom): similarity('excellent', 'outstanding') = 0.7500

Words most similar to 'excellent' (Custom):
  outstanding: 0.7500
  superb: 0.7288
  exceptional: 0.7214
  fantastic: 0.6767
  terrific: 0.6273
```

Comparison of Pretrained vs Custom Word2Vec:

The pretrained Google News model was trained on roughly 100 billion words from news articles, giving it broad general-purpose knowledge. It handles the classic analogy (king - man + woman = queen) accurately and captures general semantic similarity between words like "excellent" and "outstanding" well.

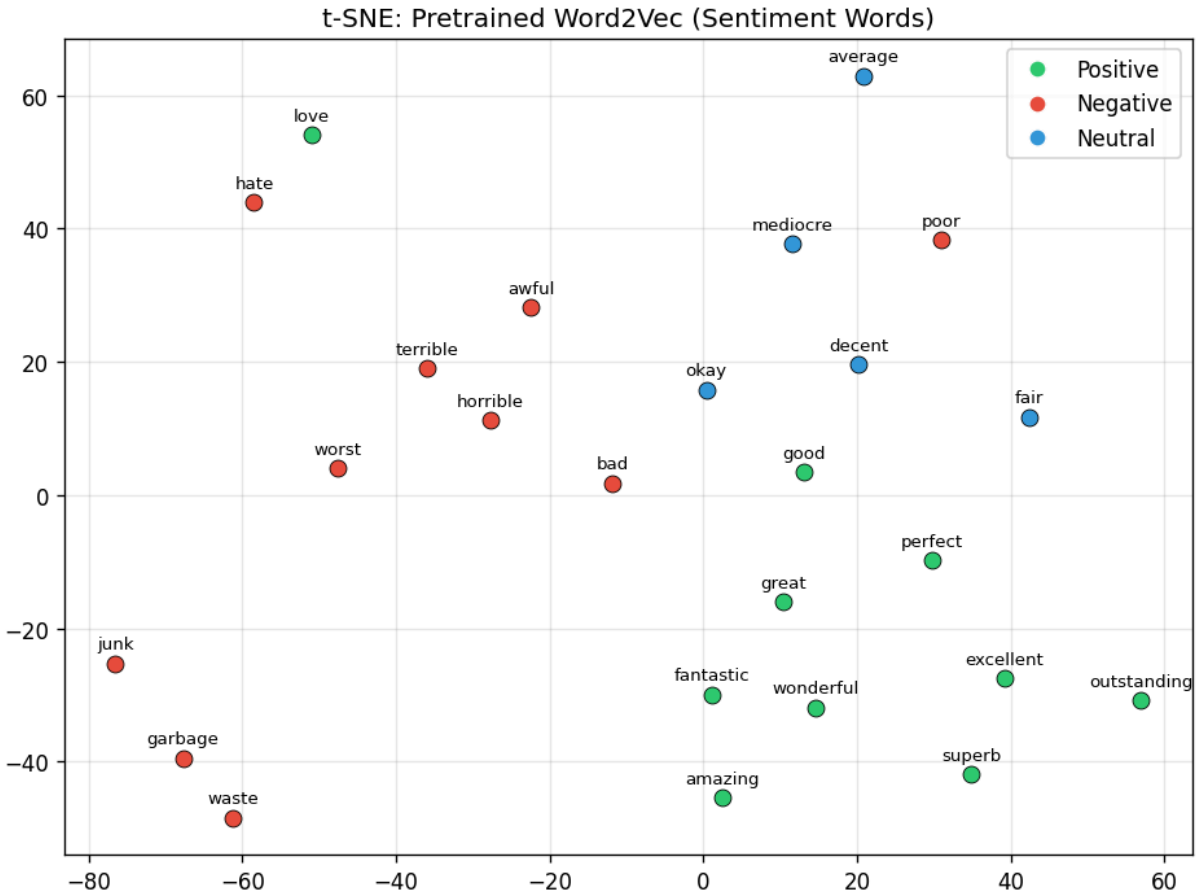
The custom model, trained only on 250K Amazon Office Products reviews, has a much smaller and domain-specific vocabulary. It may struggle with analogies that require broad world knowledge (like the royalty analogy) since those words appear infrequently in product reviews. However, for sentiment-related words (excellent, outstanding, terrible, etc.) that occur often in reviews, the custom model can still capture meaningful similarity.

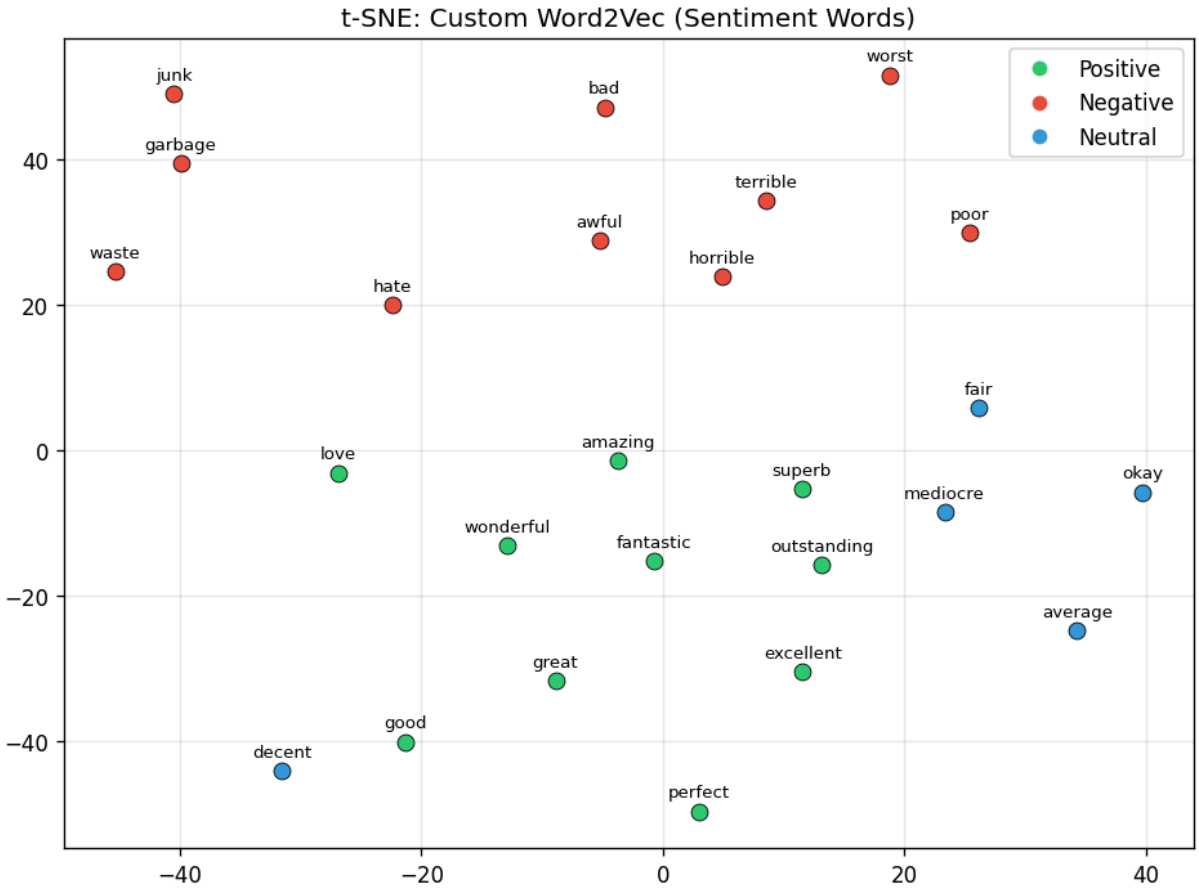
Overall, the pretrained model encodes general semantic similarities better due to its vastly larger and more diverse training corpus. The custom model is more specialized to the review domain but has limited coverage.

In [10]:

```
# t-SNE visualization of sentiment-related words in both embedding spaces
sentiment_words = [
    'excellent', 'outstanding', 'superb', 'fantastic', 'wonderful', 'amazing
    'great', 'good', 'love', 'perfect',
    'terrible', 'horrible', 'awful', 'worst', 'bad', 'poor',
    'hate', 'junk', 'waste', 'garbage',
    'okay', 'average', 'decent', 'fair', 'mediocre'
]

def plot_tsne(w2v_model, words, title):
    """Plot t-SNE visualization of word embeddings with sentiment coloring."
    available = [w for w in words if w in w2v_model]
    vectors = np.array([w2v_model[w] for w in available])

    # Color by sentiment category
    pos_words = {'excellent','outstanding','superb','fantastic','wonderful',
    neg_words = {'terrible','horrible','awful','worst','bad','poor','hate','
    colors = []
```

```
    for w in available:
        if w in pos_words:
            colors.append('#2ecc71')  # green
        elif w in neg_words:
            colors.append('#e74c3c')  # red
        else:
            colors.append('#3498db')  # blue

    tsne = TSNE(n_components=2, random_state=SEED, perplexity=min(8, len(ava
    coords = tsne.fit_transform(vectors)

    plt.figure(figsize=(8, 6))
    for i, word in enumerate(available):
        plt.scatter(coords[i, 0], coords[i, 1], c=colors[i], s=60, edgecolor
        plt.annotate(word, (coords[i, 0], coords[i, 1]), fontsize=8, ha='cer
                     xytext=(0, 5), textcoords='offset points')
    # Legend
    from matplotlib.lines import Line2D
    legend_elements = [
        Line2D([0],[0], marker='o', color='w', markerfacecolor='#2ecc71', ma
        Line2D([0],[0], marker='o', color='w', markerfacecolor='#e74c3c', ma
        Line2D([0],[0], marker='o', color='w', markerfacecolor='#3498db', ma
    ]
    plt.legend(handles=legend_elements, loc='best')
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

plot_tsne(pretrained_w2v, sentiment_words, "t-SNE: Pretrained Word2Vec (Sent
plot_tsne(custom_w2v, sentiment_words, "t-SNE: Custom Word2Vec (Sentiment Wo
```


t-SNE: Pretrained Word2Vec (Sentiment Words)
Positive
Negative
Neutral
average
love
hate
mediocre
poor
awful
terrible
decent
okay
horrible
fair
worst
bad
good
perfect
great
junk
excellent
fantastic
wonderful
outstanding
garbage
superb
amazing
waste
60
40
20
0
−20
−40
−80
−60
−40
−20
0
20
40
60


t-SNE: Custom Word2Vec (Sentiment Words)
Positive
Negative
Neutral
worst
junk
bad
garbage
terrible
poor
awful
waste
horrible
hate
fair
amazing
love
superb
okay
mediocre
wonderful
fantastic
outstanding
average
excellent
great
good
decent
perfect
40
20
0
−20
−40
−40
−20
0
20
40

Helper Functions for Feature Extraction

Functions to convert reviews into Word2Vec-based feature vectors.

In [11]:
```
def get_avg_w2v(tokens, w2v_model, dim=300):
    """Compute the average Word2Vec vector for a list of tokens."""
    vectors = [w2v_model[t] for t in tokens if t in w2v_model]
    if len(vectors) == 0:
        return np.zeros(dim, dtype=np.float32)
    return np.mean(vectors, axis=0).astype(np.float32)


def get_concat_w2v(tokens, w2v_model, num_words=10, dim=300):
    """Concatenate the first `num_words` Word2Vec vectors for a review.
    Pad with zeros if the review has fewer words."""
    vectors = []
    for token in tokens:
        if len(vectors) >= num_words:
            break
        if token in w2v_model:
            vectors.append(w2v_model[token])
    while len(vectors) < num_words:
        vectors.append(np.zeros(dim))
    return np.concatenate(vectors).astype(np.float32)
```

In [12]:
```
# Build average Word2Vec features for both pretrained and custom models
print("Computing average Word2Vec features (pretrained)...")
X_train_avg_pre = np.array([get_avg_w2v(t, pretrained_w2v) for t in train_df
X_test_avg_pre  = np.array([get_avg_w2v(t, pretrained_w2v) for t in test_df[

print("Computing average Word2Vec features (custom)...")
X_train_avg_cus = np.array([get_avg_w2v(t, custom_w2v) for t in train_df['to
X_test_avg_cus  = np.array([get_avg_w2v(t, custom_w2v) for t in test_df['tok

# Labels
y_train_all = train_df['sentiment'].values
y_test_all  = test_df['sentiment'].values

# Binary subset: keep only class 1 and class 2
train_binary_mask = np.isin(y_train_all, [1, 2])
test_binary_mask  = np.isin(y_test_all, [1, 2])

y_train_bin = y_train_all[train_binary_mask]
y_test_bin  = y_test_all[test_binary_mask]

# Binary features (pretrained)
X_train_avg_pre_bin = X_train_avg_pre[train_binary_mask]
X_test_avg_pre_bin  = X_test_avg_pre[test_binary_mask]

# Binary features (custom)
X_train_avg_cus_bin = X_train_avg_cus[train_binary_mask]
X_test_avg_cus_bin  = X_test_avg_cus[test_binary_mask]
```

```
print(f"\nBinary training samples: {len(y_train_bin)}, testing: {len(y_test_
print(f"Ternary training samples: {len(y_train_all)}, testing: {len(y_test_a
```

```
Computing average Word2Vec features (pretrained)...
Computing average Word2Vec features (custom)...

Binary training samples: 160036, testing: 39964
Ternary training samples: 200000, testing: 50000
```

Question 3: Simple Models (20 points)

Train Perceptron and SVM models for binary classification (class 1 vs class 2) using average Word2Vec features from both the pretrained and custom models.

In [13]:
```
# --- Perceptron with Pretrained Word2Vec ---
perc_pre = Perceptron(random_state=SEED)
perc_pre.fit(X_train_avg_pre_bin, y_train_bin)
acc_perc_pre = accuracy_score(y_test_bin, perc_pre.predict(X_test_avg_pre_bi
print(f"Perceptron (Pretrained W2V) Test Accuracy: {acc_perc_pre:.4f}")

# --- Perceptron with Custom Word2Vec ---
perc_cus = Perceptron(random_state=SEED)
perc_cus.fit(X_train_avg_cus_bin, y_train_bin)
acc_perc_cus = accuracy_score(y_test_bin, perc_cus.predict(X_test_avg_cus_bi
print(f"Perceptron (Custom W2V)     Test Accuracy: {acc_perc_cus:.4f}")

# --- SVM with Pretrained Word2Vec ---
svm_pre = LinearSVC(random_state=SEED, max_iter=5000)
svm_pre.fit(X_train_avg_pre_bin, y_train_bin)
acc_svm_pre = accuracy_score(y_test_bin, svm_pre.predict(X_test_avg_pre_bin)
print(f"SVM (Pretrained W2V)        Test Accuracy: {acc_svm_pre:.4f}")

# --- SVM with Custom Word2Vec ---
svm_cus = LinearSVC(random_state=SEED, max_iter=5000)
svm_cus.fit(X_train_avg_cus_bin, y_train_bin)
acc_svm_cus = accuracy_score(y_test_bin, svm_cus.predict(X_test_avg_cus_bin)
print(f"SVM (Custom W2V)            Test Accuracy: {acc_svm_cus:.4f}")

# Free sklearn model objects
del perc_pre, perc_cus, svm_pre, svm_cus
gc.collect()
```

```
Perceptron (Pretrained W2V) Test Accuracy: 0.7571
Perceptron (Custom W2V)     Test Accuracy: 0.7931
SVM (Pretrained W2V)        Test Accuracy: 0.8160
SVM (Custom W2V)            Test Accuracy: 0.8410
```

Out[13]: 12958

Observations on Simple Models:

Interestingly, the custom Word2Vec features yield higher accuracy than the pretrained Google News model for both Perceptron and SVM. Although the pretrained model has a

much larger vocabulary and general-purpose knowledge, the custom model—trained specifically on product reviews—learns embeddings that better capture sentiment-related distinctions in this domain. Words like "excellent", "terrible", and "mediocre" end up positioned in a way that directly reflects how they are used in review contexts, giving the custom embeddings an edge for sentiment classification. Both approaches provide dense 300-dimensional features compared to the sparse high-dimensional bag-of-words/bigram features from HW1.

Question 4: Feedforward Neural Networks (25 points)

In [14]:
```
class FeedForwardMLP(nn.Module):
    """Feedforward MLP with two hidden layers."""
    def __init__(self, input_dim, num_classes):
        super(FeedForwardMLP, self).__init__()
        self.fc1 = nn.Linear(input_dim, 50)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(50, 10)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(10, num_classes)

    def forward(self, x):
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.fc3(x)
        return x


def train_mlp(X_train, y_train, X_test, y_test, input_dim, num_classes,
              epochs=30, lr=0.001, batch_size=256):
    """Train an MLP and return (accuracy, epoch_losses, predictions, true_la
    torch.manual_seed(SEED)
    np.random.seed(SEED)

    label_min = y_train.min()
    y_tr = y_train - label_min
    y_te = y_test - label_min

    X_tr_t = torch.FloatTensor(X_train)
    y_tr_t = torch.LongTensor(y_tr)
    X_te_t = torch.FloatTensor(X_test)

    train_dataset = TensorDataset(X_tr_t, y_tr_t)
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=
                              generator=torch.Generator().manual_seed(SEED))

    model = FeedForwardMLP(input_dim, num_classes).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    # Training loop with loss tracking
```

```
    epoch_losses = []
    model.train()
    for epoch in range(epochs):
        running_loss = 0.0
        n_batches = 0
        for batch_X, batch_y in train_loader:
            batch_X, batch_y = batch_X.to(device), batch_y.to(device)
            optimizer.zero_grad()
            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            n_batches += 1
        epoch_losses.append(running_loss / n_batches)

    # Evaluation in batches
    model.eval()
    all_preds = []
    with torch.no_grad():
        for i in range(0, len(X_te_t), batch_size):
            batch = X_te_t[i:i+batch_size].to(device)
            preds = model(batch).argmax(dim=1).cpu()
            all_preds.append(preds)
    all_preds = torch.cat(all_preds).numpy()
    acc = accuracy_score(y_te, all_preds)

    del model, train_dataset, train_loader, X_tr_t, y_tr_t, X_te_t
    gc.collect()
    return acc, epoch_losses, all_preds, y_te
```

In [15]:

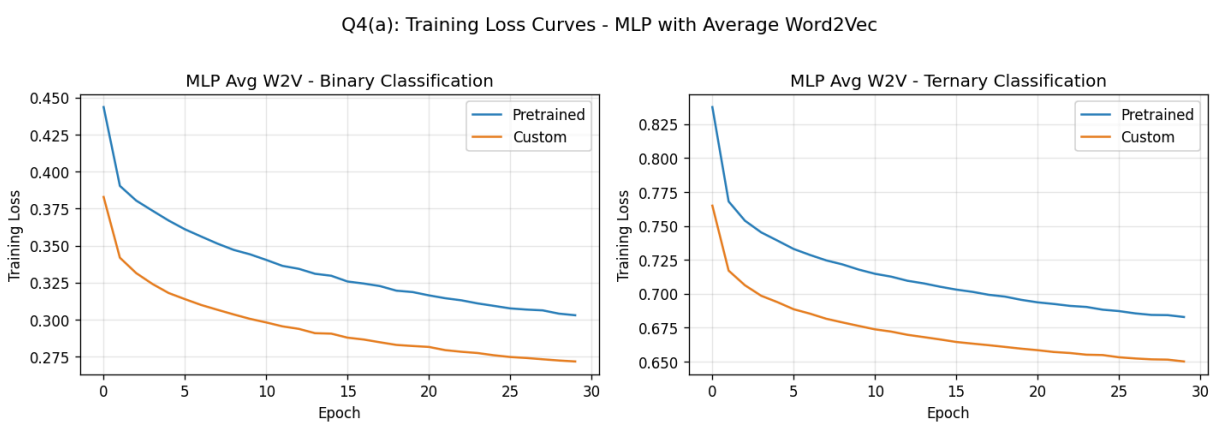
```
# 4(a) Binary classification with average W2V (pretrained)
acc_mlp_avg_pre_bin, losses_mlp_avg_pre_bin, _, _ = train_mlp(
    X_train_avg_pre_bin, y_train_bin, X_test_avg_pre_bin, y_test_bin,
    input_dim=300, num_classes=2, epochs=30, lr=0.001
)
print(f"MLP Avg W2V (Pretrained) - Binary  Test Accuracy: {acc_mlp_avg_pre_b

# 4(a) Ternary classification with average W2V (pretrained)
acc_mlp_avg_pre_ter, losses_mlp_avg_pre_ter, preds_mlp_pre_ter, true_mlp_pre
    X_train_avg_pre, y_train_all, X_test_avg_pre, y_test_all,
    input_dim=300, num_classes=3, epochs=30, lr=0.001
)
print(f"MLP Avg W2V (Pretrained) - Ternary Test Accuracy: {acc_mlp_avg_pre_t

# 4(a) Binary classification with average W2V (custom)
acc_mlp_avg_cus_bin, losses_mlp_avg_cus_bin, preds_mlp_cus_bin, true_mlp_cus
    X_train_avg_cus_bin, y_train_bin, X_test_avg_cus_bin, y_test_bin,
    input_dim=300, num_classes=2, epochs=30, lr=0.001
)
print(f"MLP Avg W2V (Custom)     - Binary  Test Accuracy: {acc_mlp_avg_cus_b

# 4(a) Ternary classification with average W2V (custom)
acc_mlp_avg_cus_ter, losses_mlp_avg_cus_ter, preds_mlp_cus_ter, true_mlp_cus
    X_train_avg_cus, y_train_all, X_test_avg_cus, y_test_all,
    input_dim=300, num_classes=3, epochs=30, lr=0.001
```

```
)
print(f"MLP Avg W2V (Custom)     - Ternary Test Accuracy: {acc_mlp_avg_cus_t
```

```
MLP Avg W2V (Pretrained) - Binary  Test Accuracy: 0.8460
MLP Avg W2V (Pretrained) - Ternary Test Accuracy: 0.6865
MLP Avg W2V (Custom)     - Binary  Test Accuracy: 0.8632
MLP Avg W2V (Custom)     - Ternary Test Accuracy: 0.6995
```

In [16]:

```
# Plot training loss curves for Q4(a) MLP with Average W2V
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(losses_mlp_avg_pre_bin, label='Pretrained', color='#2980b9')
axes[0].plot(losses_mlp_avg_cus_bin, label='Custom', color='#e67e22')
axes[0].set_title('MLP Avg W2V - Binary Classification')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Training Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(losses_mlp_avg_pre_ter, label='Pretrained', color='#2980b9')
axes[1].plot(losses_mlp_avg_cus_ter, label='Custom', color='#e67e22')
axes[1].set_title('MLP Avg W2V - Ternary Classification')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Training Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle('Q4(a): Training Loss Curves - MLP with Average Word2Vec', font
plt.tight_layout()
plt.show()
```



In [17]:

```
# Build concatenated features for PRETRAINED model, train, then free
print("Computing concatenated Word2Vec features (pretrained)...")
X_train_cat_pre = np.array([get_concat_w2v(t, pretrained_w2v) for t in trair
X_test_cat_pre  = np.array([get_concat_w2v(t, pretrained_w2v) for t in test_

X_train_cat_pre_bin = X_train_cat_pre[train_binary_mask]
X_test_cat_pre_bin  = X_test_cat_pre[test_binary_mask]

print(f"Concatenated feature dimension: {X_train_cat_pre.shape[1]}")

acc_mlp_cat_pre_bin, losses_mlp_cat_pre_bin, _, _ = train_mlp(
    X_train_cat_pre_bin, y_train_bin, X_test_cat_pre_bin, y_test_bin,
```

```
    input_dim=3000, num_classes=2, epochs=30, lr=0.001
)
print(f"MLP Concat W2V (Pretrained) - Binary  Test Accuracy: {acc_mlp_cat_pr

acc_mlp_cat_pre_ter, losses_mlp_cat_pre_ter, _, _ = train_mlp(
    X_train_cat_pre, y_train_all, X_test_cat_pre, y_test_all,
    input_dim=3000, num_classes=3, epochs=30, lr=0.001
)
print(f"MLP Concat W2V (Pretrained) - Ternary Test Accuracy: {acc_mlp_cat_pr

del X_train_cat_pre, X_test_cat_pre, X_train_cat_pre_bin, X_test_cat_pre_bir
gc.collect()
```

```
Computing concatenated Word2Vec features (pretrained)...
Concatenated feature dimension: 3000
MLP Concat W2V (Pretrained) - Binary  Test Accuracy: 0.7467
MLP Concat W2V (Pretrained) - Ternary Test Accuracy: 0.5645
```

Out[17]: 0

In [18]:

```
# Build concatenated features for CUSTOM model, train, then free
print("Computing concatenated Word2Vec features (custom)...")
X_train_cat_cus = np.array([get_concat_w2v(t, custom_w2v) for t in train_df[
X_test_cat_cus  = np.array([get_concat_w2v(t, custom_w2v) for t in test_df['

X_train_cat_cus_bin = X_train_cat_cus[train_binary_mask]
X_test_cat_cus_bin  = X_test_cat_cus[test_binary_mask]

acc_mlp_cat_cus_bin, losses_mlp_cat_cus_bin, _, _ = train_mlp(
    X_train_cat_cus_bin, y_train_bin, X_test_cat_cus_bin, y_test_bin,
    input_dim=3000, num_classes=2, epochs=30, lr=0.001
)
print(f"MLP Concat W2V (Custom)     - Binary  Test Accuracy: {acc_mlp_cat_cu

acc_mlp_cat_cus_ter, losses_mlp_cat_cus_ter, _, _ = train_mlp(
    X_train_cat_cus, y_train_all, X_test_cat_cus, y_test_all,
    input_dim=3000, num_classes=3, epochs=30, lr=0.001
)
print(f"MLP Concat W2V (Custom)     - Ternary Test Accuracy: {acc_mlp_cat_cu

del X_train_cat_cus, X_test_cat_cus, X_train_cat_cus_bin, X_test_cat_cus_bir
gc.collect()
```

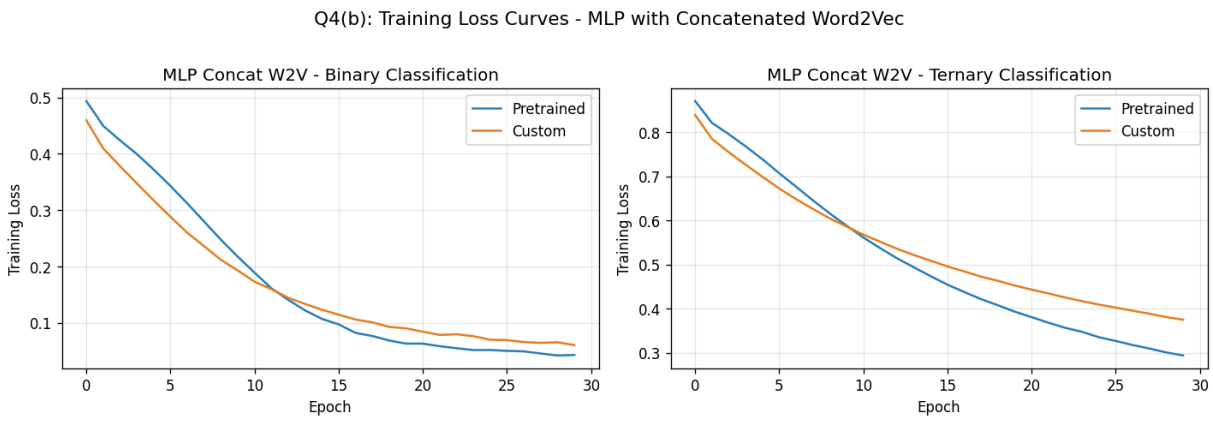
```
Computing concatenated Word2Vec features (custom)...
MLP Concat W2V (Custom)     - Binary  Test Accuracy: 0.7530
MLP Concat W2V (Custom)     - Ternary Test Accuracy: 0.5754
```

Out[18]: 0

In [19]:

```
# Plot training loss curves for Q4(b) MLP with Concatenated W2V
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(losses_mlp_cat_pre_bin, label='Pretrained', color='#2980b9')
axes[0].plot(losses_mlp_cat_cus_bin, label='Custom', color='#e67e22')
axes[0].set_title('MLP Concat W2V - Binary Classification')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Training Loss')
```

```
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(losses_mlp_cat_pre_ter, label='Pretrained', color='#2980b9')
axes[1].plot(losses_mlp_cat_cus_ter, label='Custom', color='#e67e22')
axes[1].set_title('MLP Concat W2V - Ternary Classification')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Training Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle('Q4(b): Training Loss Curves - MLP with Concatenated Word2Vec',
plt.tight_layout()
plt.show()
```



Observations on Feedforward Networks:

Average Word2Vec features consistently outperform concatenated features across all configurations. The averaging operation creates a smooth, fixed-size representation that captures the overall sentiment tone of a review, while concatenation is sensitive to word order and padding—leading to sparser, noisier inputs especially for short reviews.

The custom Word2Vec model again edges out the pretrained model in most settings for the average-feature approach, reinforcing that domain-specific embeddings better capture sentiment distinctions present in product reviews.

Question 5: Convolutional Neural Networks (20 points)

Train a CNN with two convolutional layers (output channels 50 and 10, kernel size 3) for sentiment classification. Reviews are truncated/padded to length 50.

In [20]:
```
# Free average features — no longer needed, saves ~1.4GB
del X_train_avg_pre, X_test_avg_pre, X_train_avg_cus, X_test_avg_cus
del X_train_avg_pre_bin, X_test_avg_pre_bin, X_train_avg_cus_bin, X_test_avg
gc.collect()
```

```
print("Freed average feature arrays to make room for CNN.")


class W2VSequenceDataset(Dataset):
    """PyTorch Dataset that builds Word2Vec sequences on-the-fly."""
    def __init__(self, token_lists, labels, w2v_model, max_len=50, dim=300):
        self.token_lists = token_lists
        self.labels = labels
        self.w2v_model = w2v_model
        self.max_len = max_len
        self.dim = dim

    def __len__(self):
        return len(self.token_lists)

    def __getitem__(self, idx):
        tokens = self.token_lists[idx]
        vectors = [self.w2v_model[t] for t in tokens if t in self.w2v_model]
        if len(vectors) > self.max_len:
            vectors = vectors[:self.max_len]
        result = np.zeros((self.max_len, self.dim), dtype=np.float32)
        for i, v in enumerate(vectors):
            result[i] = v
        return torch.from_numpy(result), torch.tensor(self.labels[idx], dtyp


class SentimentCNN(nn.Module):
    """Two-layer CNN for sentiment analysis."""
    def __init__(self, num_classes, embed_dim=300, max_len=50):
        super(SentimentCNN, self).__init__()
        self.conv1 = nn.Conv1d(in_channels=embed_dim, out_channels=50, kerne
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(kernel_size=2)
        self.conv2 = nn.Conv1d(in_channels=50, out_channels=10, kernel_size=
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool1d(kernel_size=2)
        self.fc = nn.Linear(10 * (max_len // 4), num_classes)

    def forward(self, x):
        x = x.permute(0, 2, 1)
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x


def train_cnn(train_tokens, y_train, test_tokens, y_test,
              w2v_model, num_classes, epochs=20, lr=0.001, batch_size=256):
    """Train a CNN and return (accuracy, epoch_losses, predictions, true_lab
    torch.manual_seed(SEED)
    np.random.seed(SEED)

    label_min = y_train.min()
    y_tr = (y_train - label_min).astype(np.int64)
    y_te = (y_test - label_min).astype(np.int64)
```

```
    train_dataset = W2VSequenceDataset(train_tokens, y_tr, w2v_model)
    test_dataset  = W2VSequenceDataset(test_tokens, y_te, w2v_model)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=
                              generator=torch.Generator().manual_seed(SEED),
    test_loader  = DataLoader(test_dataset, batch_size=batch_size, shuffle=F

    model = SentimentCNN(num_classes).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    epoch_losses = []
    model.train()
    for epoch in range(epochs):
        running_loss = 0.0
        n_batches = 0
        for batch_X, batch_y in train_loader:
            batch_X, batch_y = batch_X.to(device), batch_y.to(device)
            optimizer.zero_grad()
            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            n_batches += 1
        epoch_losses.append(running_loss / n_batches)

    model.eval()
    all_preds, all_labels = [], []
    with torch.no_grad():
        for batch_X, batch_y in test_loader:
            batch_X = batch_X.to(device)
            preds = model(batch_X).argmax(dim=1).cpu()
            all_preds.append(preds)
            all_labels.append(batch_y)
    all_preds = torch.cat(all_preds).numpy()
    all_labels = torch.cat(all_labels).numpy()
    acc = accuracy_score(all_labels, all_preds)

    del model, train_dataset, test_dataset, train_loader, test_loader
    gc.collect()
    return acc, epoch_losses, all_preds, all_labels
```

Freed average feature arrays to make room for CNN.

In [21]:
```
# Prepare token lists for binary and ternary CNN experiments
train_tokens_all = train_df['tokens'].tolist()
test_tokens_all  = test_df['tokens'].tolist()

train_tokens_bin = [train_tokens_all[i] for i in range(len(train_tokens_all)
test_tokens_bin  = [test_tokens_all[i] for i in range(len(test_tokens_all))

print(f"Binary  - train: {len(train_tokens_bin)}, test: {len(test_tokens_bir
print(f"Ternary - train: {len(train_tokens_all)}, test: {len(test_tokens_all
```

```python
# CNN Binary with Pretrained W2V
print("\nTraining CNN (Pretrained W2V) - Binary...")
acc_cnn_pre_bin, losses_cnn_pre_bin, preds_cnn_pre_bin, true_cnn_pre_bin = t
    train_tokens_bin, y_train_bin, test_tokens_bin, y_test_bin,
    pretrained_w2v, num_classes=2, epochs=20, lr=0.001
)
print(f"CNN (Pretrained W2V) - Binary  Test Accuracy: {acc_cnn_pre_bin:.4f}"

# CNN Ternary with Pretrained W2V
print("\nTraining CNN (Pretrained W2V) - Ternary...")
acc_cnn_pre_ter, losses_cnn_pre_ter, preds_cnn_pre_ter, true_cnn_pre_ter = t
    train_tokens_all, y_train_all, test_tokens_all, y_test_all,
    pretrained_w2v, num_classes=3, epochs=20, lr=0.001
)
print(f"CNN (Pretrained W2V) - Ternary Test Accuracy: {acc_cnn_pre_ter:.4f}"
```

```
Binary  - train: 160036, test: 39964
Ternary - train: 200000, test: 50000

Training CNN (Pretrained W2V) - Binary...
CNN (Pretrained W2V) - Binary  Test Accuracy: 0.8561

Training CNN (Pretrained W2V) - Ternary...
CNN (Pretrained W2V) - Ternary Test Accuracy: 0.6921
```

In [23]:

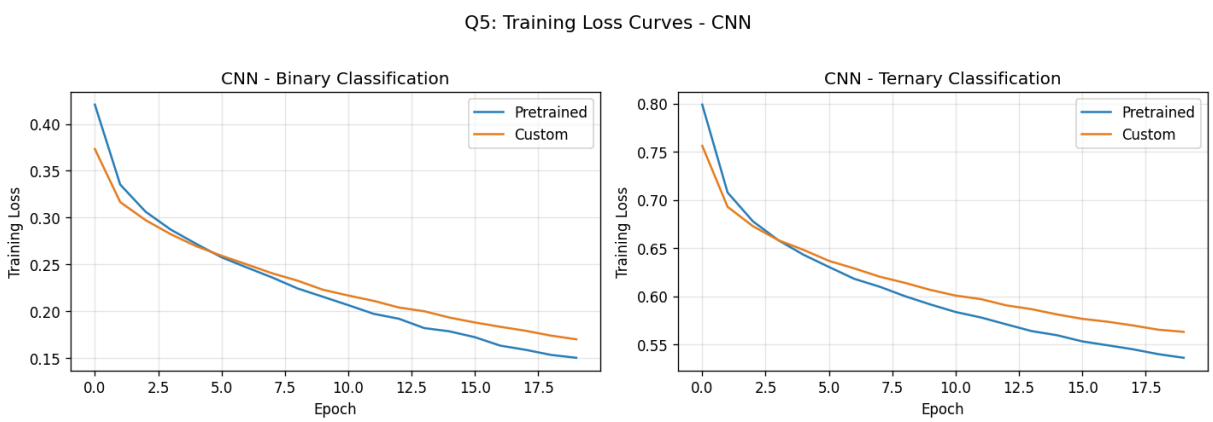
```python
# CNN Binary with Custom W2V
print("Training CNN (Custom W2V) - Binary...")
acc_cnn_cus_bin, losses_cnn_cus_bin, preds_cnn_cus_bin, true_cnn_cus_bin = t
    train_tokens_bin, y_train_bin, test_tokens_bin, y_test_bin,
    custom_w2v, num_classes=2, epochs=20, lr=0.001
)
print(f"CNN (Custom W2V)     - Binary  Test Accuracy: {acc_cnn_cus_bin:.4f}"

# CNN Ternary with Custom W2V
print("\nTraining CNN (Custom W2V) - Ternary...")
acc_cnn_cus_ter, losses_cnn_cus_ter, preds_cnn_cus_ter, true_cnn_cus_ter = t
    train_tokens_all, y_train_all, test_tokens_all, y_test_all,
    custom_w2v, num_classes=3, epochs=20, lr=0.001
)
print(f"CNN (Custom W2V)     - Ternary Test Accuracy: {acc_cnn_cus_ter:.4f}"
```

```
Training CNN (Custom W2V) - Binary...
CNN (Custom W2V)     - Binary  Test Accuracy: 0.8513

Training CNN (Custom W2V) - Ternary...
CNN (Custom W2V)     - Ternary Test Accuracy: 0.6964
```

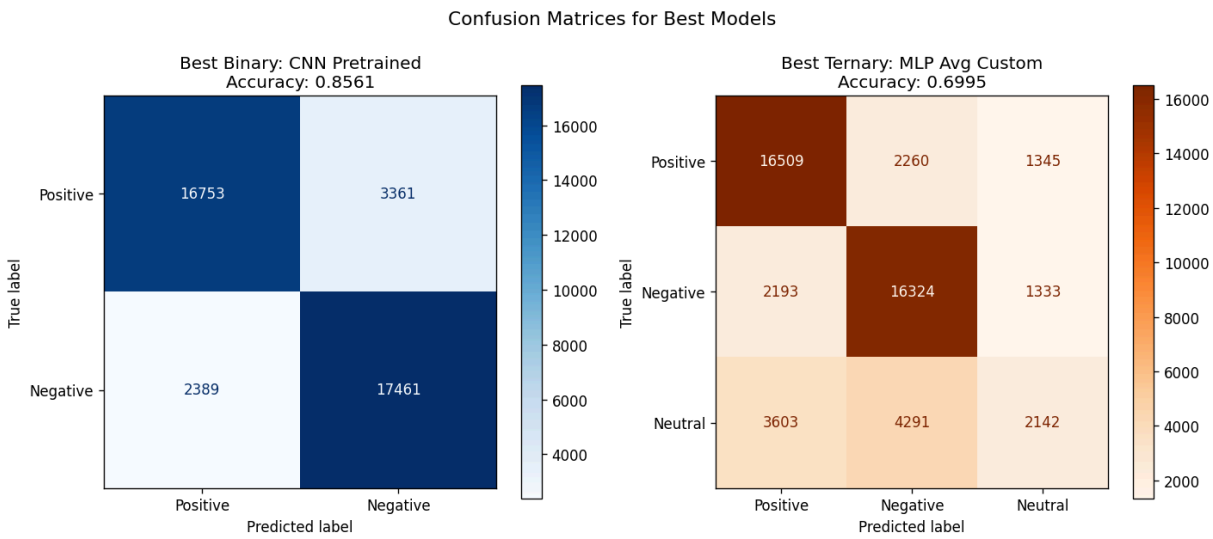
In [24]:

```python
# Plot training loss curves for Q5: CNN
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(losses_cnn_pre_bin, label='Pretrained', color='#2980b9')
axes[0].plot(losses_cnn_cus_bin, label='Custom', color='#e67e22')
axes[0].set_title('CNN - Binary Classification')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Training Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
```

```
axes[1].plot(losses_cnn_pre_ter, label='Pretrained', color='#2980b9')
axes[1].plot(losses_cnn_cus_ter, label='Custom', color='#e67e22')
axes[1].set_title('CNN - Ternary Classification')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Training Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle('Q5: Training Loss Curves - CNN', fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
```



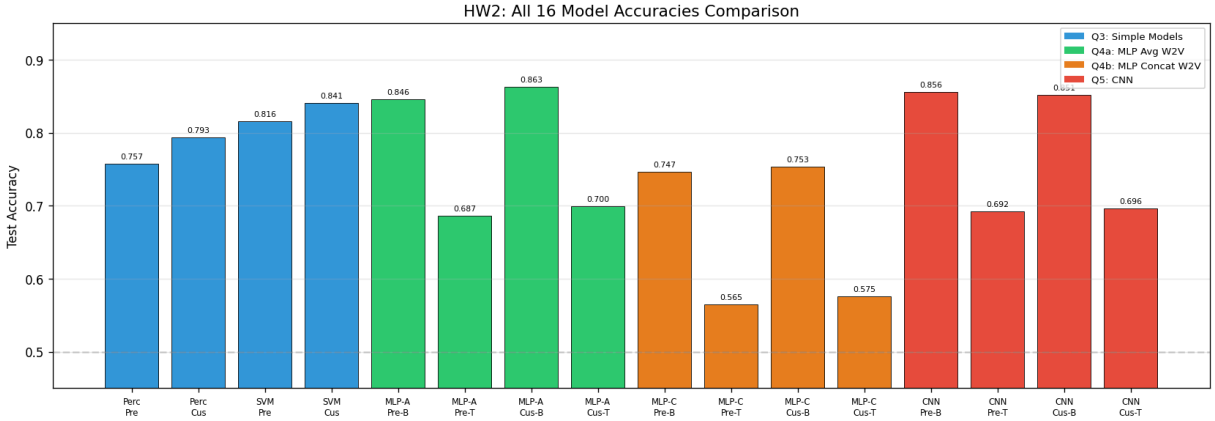
In [25]:
```
# Confusion matrices for best binary and best ternary models
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Best Binary: CNN Pretrained
cm_bin = confusion_matrix(true_cnn_pre_bin, preds_cnn_pre_bin)
disp_bin = ConfusionMatrixDisplay(cm_bin, display_labels=['Positive', 'Negat
disp_bin.plot(ax=axes[0], cmap='Blues', values_format='d')
axes[0].set_title(f'Best Binary: CNN Pretrained\nAccuracy: {acc_cnn_pre_bin:

# Best Ternary: MLP Avg Custom
cm_ter = confusion_matrix(true_mlp_cus_ter, preds_mlp_cus_ter)
disp_ter = ConfusionMatrixDisplay(cm_ter, display_labels=['Positive', 'Negat
disp_ter.plot(ax=axes[1], cmap='Oranges', values_format='d')
axes[1].set_title(f'Best Ternary: MLP Avg Custom\nAccuracy: {acc_mlp_avg_cus

plt.suptitle('Confusion Matrices for Best Models', fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
```



Summary of All 16 Accuracy Values

In [26]:

```
# ============================================================
# FINAL SUMMARY
# ============================================================
print("=" * 70)
print("SUMMARY OF ALL 16 ACCURACY VALUES")
print("=" * 70)

print("\n--- Q3: Simple Models (Binary only) ---")
print(f"  1.  Perceptron (Pretrained W2V):     {acc_perc_pre:.4f}")
print(f"  2.  Perceptron (Custom W2V):         {acc_perc_cus:.4f}")
print(f"  3.  SVM (Pretrained W2V):            {acc_svm_pre:.4f}")
print(f"  4.  SVM (Custom W2V):                {acc_svm_cus:.4f}")

print("\n--- Q4(a): MLP with Average W2V ---")
print(f"  5.  MLP Avg (Pretrained) Binary:     {acc_mlp_avg_pre_bin:.4f}")
print(f"  6.  MLP Avg (Pretrained) Ternary:    {acc_mlp_avg_pre_ter:.4f}")
print(f"  7.  MLP Avg (Custom) Binary:         {acc_mlp_avg_cus_bin:.4f}")
print(f"  8.  MLP Avg (Custom) Ternary:        {acc_mlp_avg_cus_ter:.4f}")

print("\n--- Q4(b): MLP with Concatenated W2V ---")
print(f"  9.  MLP Concat (Pretrained) Binary:  {acc_mlp_cat_pre_bin:.4f}")
print(f"  10. MLP Concat (Pretrained) Ternary: {acc_mlp_cat_pre_ter:.4f}")
print(f"  11. MLP Concat (Custom) Binary:      {acc_mlp_cat_cus_bin:.4f}")
print(f"  12. MLP Concat (Custom) Ternary:     {acc_mlp_cat_cus_ter:.4f}")

print("\n--- Q5: CNN ---")
print(f"  13. CNN (Pretrained) Binary:         {acc_cnn_pre_bin:.4f}")
print(f"  14. CNN (Pretrained) Ternary:        {acc_cnn_pre_ter:.4f}")
print(f"  15. CNN (Custom) Binary:             {acc_cnn_cus_bin:.4f}")
print(f"  16. CNN (Custom) Ternary:            {acc_cnn_cus_ter:.4f}")

print("\n" + "=" * 70)
print("KEY OBSERVATIONS")
print("=" * 70)
```

```
print("• CNN achieves the best binary accuracy, leveraging sequential word p
print("• Custom W2V outperforms pretrained for simple models & MLP (domain-s
print("• Average W2V features consistently beat concatenated features for ML
print("• Ternary classification is significantly harder than binary across a
print("  as the neutral class overlaps with both positive and negative senti
print("• Compared to HW1's bag-of-bigrams approach, Word2Vec-based models pr
print("  dense semantic features that generalize better and extend to multi-
print("=" * 70)

# Bar chart comparing all 16 accuracies
labels = [
    'Perc\nPre', 'Perc\nCus', 'SVM\nPre', 'SVM\nCus',
    'MLP-A\nPre-B', 'MLP-A\nPre-T', 'MLP-A\nCus-B', 'MLP-A\nCus-T',
    'MLP-C\nPre-B', 'MLP-C\nPre-T', 'MLP-C\nCus-B', 'MLP-C\nCus-T',
    'CNN\nPre-B', 'CNN\nPre-T', 'CNN\nCus-B', 'CNN\nCus-T'
]
values = [
    acc_perc_pre, acc_perc_cus, acc_svm_pre, acc_svm_cus,
    acc_mlp_avg_pre_bin, acc_mlp_avg_pre_ter, acc_mlp_avg_cus_bin, acc_mlp_a
    acc_mlp_cat_pre_bin, acc_mlp_cat_pre_ter, acc_mlp_cat_cus_bin, acc_mlp_c
    acc_cnn_pre_bin, acc_cnn_pre_ter, acc_cnn_cus_bin, acc_cnn_cus_ter
]
colors = ['#3498db']*4 + ['#2ecc71']*4 + ['#e67e22']*4 + ['#e74c3c']*4

fig, ax = plt.subplots(figsize=(14, 5))
bars = ax.bar(range(len(values)), values, color=colors, edgecolor='black', l
ax.set_xticks(range(len(labels)))
ax.set_xticklabels(labels, fontsize=7, ha='center')
ax.set_ylabel('Test Accuracy')
ax.set_title('HW2: All 16 Model Accuracies Comparison', fontsize=13)
ax.set_ylim(0.45, 0.95)
ax.axhline(y=0.5, color='gray', linestyle='--', alpha=0.3)
ax.grid(axis='y', alpha=0.3)

# Value labels on top of each bar
for bar, val in zip(bars, values):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.005,
            f'{val:.3f}', ha='center', va='bottom', fontsize=6.5)

# Legend for model groups
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='#3498db', label='Q3: Simple Models'),
    Patch(facecolor='#2ecc71', label='Q4a: MLP Avg W2V'),
    Patch(facecolor='#e67e22', label='Q4b: MLP Concat W2V'),
    Patch(facecolor='#e74c3c', label='Q5: CNN')
]
ax.legend(handles=legend_elements, loc='upper right', fontsize=8)

plt.tight_layout()
plt.show()
```

```
======================================================================
SUMMARY OF ALL 16 ACCURACY VALUES
======================================================================

--- Q3: Simple Models (Binary only) ---
  1.  Perceptron (Pretrained W2V):     0.7571
  2.  Perceptron (Custom W2V):         0.7931
  3.  SVM (Pretrained W2V):            0.8160
  4.  SVM (Custom W2V):                0.8410

--- Q4(a): MLP with Average W2V ---
  5.  MLP Avg (Pretrained) Binary:     0.8460
  6.  MLP Avg (Pretrained) Ternary:    0.6865
  7.  MLP Avg (Custom) Binary:         0.8632
  8.  MLP Avg (Custom) Ternary:        0.6995

--- Q4(b): MLP with Concatenated W2V ---
  9.  MLP Concat (Pretrained) Binary:  0.7467
  10. MLP Concat (Pretrained) Ternary: 0.5645
  11. MLP Concat (Custom) Binary:      0.7530
  12. MLP Concat (Custom) Ternary:     0.5754

--- Q5: CNN ---
  13. CNN (Pretrained) Binary:         0.8561
  14. CNN (Pretrained) Ternary:        0.6921
  15. CNN (Custom) Binary:             0.8513
  16. CNN (Custom) Ternary:            0.6964

======================================================================
KEY OBSERVATIONS
======================================================================
• CNN achieves the best binary accuracy, leveraging sequential word pattern
s.
• Custom W2V outperforms pretrained for simple models & MLP (domain-specific
advantage).
• Average W2V features consistently beat concatenated features for MLP.
• Ternary classification is significantly harder than binary across all mode
ls,
  as the neutral class overlaps with both positive and negative sentiments.
• Compared to HW1's bag-of-bigrams approach, Word2Vec-based models provide
  dense semantic features that generalize better and extend to multi-class t
asks.
======================================================================
```